SEN0401 Special Topics: Blockchain - Midterm exam (version 2, Form B)

Form code 20023 (it must equal the form code printed on your answer sheet). Chapters 1, 2, 3, 4, 5, 6. 60 minutes. Mark every answer on the separate answer sheet (bubble grid); ONE answer per row unless the item says otherwise.

1. Match statements a) to d) with the answers: sheet rows 1a to 1d, fill the answer letter. Two answers stay unused.

a) A programmer paid 7,002 bitcoin in 2011 locked the keys on an encrypted drive, lost the password and has two guesses left.

b) On 15 August 2010 two huge outputs added up to a negative number and about 184 billion bitcoin were created in block 74,638.

c) On 7 April 2014 the lead of the reference software passed from one maintainer to the next.

d) On 10 February 2014 an exchange blamed transaction malleability for its missing bitcoin, and researchers checked a year of network traffic.

A. A reference implementation needs a team and public review, not one keyholder.

B. Bech32m closes a weakness of Bech32: an extra q before a final p no longer keeps the checksum valid.

C. A signed amount needs a range check on each output, not only on the total.

D. Coins created by a coinbase are locked for 100 blocks, so a reorganisation cannot invalidate coins already spent.

E. Memory is not a backup: a recovery code must be tested by restoring it.

F. A stated cause is tested against the data: malleability changes the txid, not the amount.

2. Lena's phone is stolen. She restores her wallet from her twelve-word recovery code on a new phone. The wallet opens with different addresses, an empty balance and none of her labels. She had set a passphrase in the old wallet and had exported her labels in the BIP329 format. What is the most complete explanation and the fix?

A. Labels are derived from the seed and were lost with the phone, while the passphrase only changes how the twelve words are spelled, so nothing else is missing.

B. A passphrase forks the wallet, so the same twelve words without it give a different wallet, and labels cannot be derived from the seed; she must enter the passphrase and import the labels file.

C. The recovery code was stored on the stolen phone, so the thief has already spent the coins and nothing can be restored.

D. The gap limit stopped the scan too early; a larger gap limit would bring back both the balance and the labels.

3. On one day a price service sees three trades of bitcoin: 1 bitcoin at 62,000 dollars, 2 bitcoin at 63,000 dollars, and 1 bitcoin at a price that is not shown. The service publishes a volume-weighted figure for the day of 63,250 dollars per bitcoin. At what price, in dollars per bitcoin, did the third trade happen?
Sheet row 3, one digit per box. Unit: dollars per bitcoin.

4. A node reads the configuration file and is started with the command line shown. What happens?

```
# bitcoin.conf
prune=550
txindex=1
dbcache=450
# the command line
bitcoind -txindex=0 -dbcache=2048
```

A. It starts with pruning at 550, no transaction index and a cache of 2048: the command line overrides the file, and that removes the conflict.

B. It refuses to start: the file asks for pruning and the transaction index together, and the file is applied after the command line.

C. It starts with the transaction index on and a cache of 450, because the configuration file overrides the command line.

D. It starts with pruning and the transaction index both on, because the refusal applies only to options given on the command line.

5. A payment tracker identifies each withdrawal by its txid. Before two withdrawals confirm, a stranger changes one of them: for a legacy payment the stranger re-encodes the signature inside the input script, and for a segwit payment the stranger changes only the signature bytes in the witness. Both payments stay valid. Which outcome is right?

A. The legacy payment keeps its txid and the segwit payment gets a new one, because the txid covers the extended serialization.

B. Both payments keep their txid, because a signature is never part of any hash of the transaction.

C. Both payments get a new txid, so the tracker must follow the wtxid of every payment.

D. The legacy payment gets a new txid, so the tracker may believe it failed; the segwit payment keeps its txid, because the witness lies outside the txid.

6. A search for a vanity address needs, on average, a number of tries for one chosen letter after the leading 1. By what factor is the average number of tries larger when the search asks for three chosen letters instead of one? Sheet row 6, one digit per box. Unit: times.

7-8. Fill each blank with the two-digit CODE of the matching bank entry (sheet rows 7 to 8). Each entry is used once; some stay unused.

13 80	**21** checksum
37 128	**45** commitment
59 salt	

7. A P2PKH output holds only a 20-byte hash of the public key, which acts as a ____ to the key until the coins are spent.

8. Against collisions a 160-bit hash gives only ____ bits of work, which is why SegWit scripts use SHA256.

9. A toy chain pays a block subsidy of 40 coins. Every 50 blocks the subsidy is halved by a right shift, so any fraction is dropped, and once it reaches zero no more coins are created. How many coins does the subsidy create in total over the whole life of the chain?
Sheet row 9, one digit per box. Unit: coins.

10. The program applies two transactions to a set of unspent outputs that starts at 80 satoshis in total. What does it print, and what explains the total?

```
utxo = {'a:0': 50, 'b:1': 30}
def apply(ins, outs, txid):
    for i in ins:
        del utxo[i]
    for n, v in enumerate(outs):
        utxo[txid + ':' + str(n)] = v
apply(['a:0'], [20, 29], 'c')
apply(['c:0', 'b:1'], [45], 'd')
print(sorted(utxo), sum(utxo.values()))
```

A. ['c:1', 'd:0'] 74 : the 6 satoshis that are missing are the fees of the two transactions
B. ['a:0', 'b:1', 'c:0', 'c:1', 'd:0'] 174 : spent outputs stay in the set and every new output is added
C. ['c:1', 'd:0'] 74 : the 6 satoshis that are missing are change that was never created
D. ['c:1', 'd:0'] 80 : spending moves value to new outputs but never removes any

11. A node's configuration file holds an rpcauth line, and a P2PKH output script holds a 20-byte hash of a public key. Both keep a hash instead of the secret. Which statement compares them correctly?
A. A leak of the file reveals the password at once, because rpcauth is only an encoding; the P2PKH hash is revealed together with the key when the output is created.
B. The rpcauth hash is a commitment that the owner reveals when the coins are spent; the P2PKH hash is salted, so a leak of the script reveals no key.
C. Both hashes are salted checksums that catch typos, and both secrets are revealed as soon as a node reads the line or the script.
D. A leak of the file reveals no password, because the rpcauth hash is salted; the P2PKH hash is a commitment to the key, and the key is revealed only when the coins are spent.

12. Match statements a) to d) with the answers: sheet rows 12a to 12d, fill the answer letter. Two answers stay unused.
a) A volunteer runs a validating node on a laptop with little free disk space and does not need the old blocks.
b) A block-explorer service must be able to look up any transaction by its identifier.
c) The operator wants a message, usually by email, when the node meets an emergency.
d) A node on a thin connection must limit how many other nodes it accepts, to reduce its bandwidth use.
A. datadir
B. alertnotify
C. maxmempool
D. maxconnections
E. prune
F. txindex

13. A recovery code has 18 words. Each word carries 11 bits, and the checksum is one bit for every 32 bits of entropy. How many bits of entropy does the code hold?
Sheet row 13, one digit per box. Unit: bits.

14. In block 74,638 two outputs held 9,223,372,036,854,277,039 satoshis each. The old software checked only the total of the outputs, stored in a signed 64-bit number: it was not above the inputs or the allowed maximum, so the check passed. Which rule, shipped within hours, would have refused that block, and why?
A. Each output amount must lie between 0 and 2.1 quadrillion satoshis: a range check on every output stops a huge amount before any sum can wrap around to a negative number.
B. The txid must hash only the legacy serialization: the witness could then not hide the amounts.
C. Newly created coins must wait 100 blocks before they are spent: the 184 billion bitcoin could then not have moved.
D. The total of the outputs must not exceed the total of the inputs: the old software already checked that, and the wrapped total still passed it.

15. The program reads five sequence values. A value signals opt-in replacement when it is below 0xfffffffe, and it carries a relative timelock when bit 31 is clear. The program adds up the low 16 bits of the values that do both. What does it print?

```
seqs = [0xffffffff, 0xfffffffd, 0x0000001e,
        0x80000010, 0x00400005]
total = 0
for s in seqs:
    replace = s < 0xfffffffe
    timed = (s >> 31) == 0
    if replace and timed:
        total += s & 0xffff
print(total)
```

Printed result: sheet row 15, one digit per box.

16. Dana's coins were sent to the address that comes from the compressed public key of her private key. Her paper backup holds that private key in WIF, but this WIF has no final 01 byte. A new wallet imports it and shows a zero balance. Which diagnosis fits, and which check before funding would have caught it?
A. The missing byte changed the private key itself, so it can no longer produce her public key at all; only a second written copy of the paper would have caught it.
B. The version byte 80 marks a test network key, so the wallet looks for her coins on another network; sending a very small amount first and waiting a day would have caught it.
C. The wallet took the key as uncompressed and watches the other address of the same key; restoring from the paper and comparing the first address with the printed one, before any coins were sent, would have caught it.
D. The coins were sent to an address with no key behind it, because a valid checksum does not prove that a key exists; checking the checksum of the address again would have caught it.

17. The function cs_encode writes a count in the CompactSize encoding: a count below 253 takes 1 byte, up to 65535 takes 3 bytes, up to 4294967295 takes 5 bytes, anything larger takes 9 bytes. The program measures four counts that sit at the edges where the encoding changes length. What does it print?

```
values = [252, 253, 65536, 4294967295]
sizes = [len(cs_encode(n)) for n in values]
print(sum(sizes))
```

Printed result: sheet row 17, one digit per box.

18. Put the steps in order: sheet rows 18a to 18e, fill the place (1 = first, 5 = last) of each step.

a) A node checks the rules and relays the transaction to its peers.
b) A miner finds a valid nonce and adds a block that holds the transaction.
c) The wallet signs every input.
d) The wallet takes whole coins from its unspent outputs until they cover the payment and the fee.
e) The wallet writes the payee output and the change output and leaves the fee as the difference.

19. The chapter recomputes the white paper's table: the chance, in percent, that an attacker who holds a share of the hash power ever catches up when a payment has z confirmations. For an attacker with 10 percent: z = 1 gives 20.5, z = 2 gives 5.1, z = 3 gives 1.3 and z = 6 gives 0.02. For an attacker with 30 percent: z = 1 gives 62.8, z = 3 gives 32.5, z = 6 gives 13.2, z = 9 gives 5.5 and z = 10 gives 4.2. A shop accepts a payment only when the chance of a reversal is below 5 percent. Which statement follows from these figures?

A. The shop is equally safe at six confirmations against either attacker, because every confirmation divides the chance by the same factor.
B. The shop needs three confirmations against the 10 percent attacker but ten against the 30 percent attacker.
C. Two confirmations are enough against the 10 percent attacker, because the figure for z = 2 is below 5 percent.
D. Three confirmations are enough against the 30 percent attacker, because the chance falls below 5 percent once z reaches the attacker's share.

20. A wallet that is recovered from its code scans its keys in order and stops after gap unused keys in a row. The wallet received payments on keys 0, 3, 12 and 30. The program adds up the numbers of the keys that the scan finds. What does it print?

```
used = {0, 3, 12, 30}
gap, run, found = 10, 0, []
for i in range(40):
    if i in used:
        found.append(i)
        run = 0
    else:
        run += 1
        if run == gap:
            break
print(sum(found))
```

Printed result: sheet row 20, one digit per box.

21. The course tells two stories about the exchange Mt. Gox in 2014. On 28 February it filed for bankruptcy after admitting that about 850,000 bitcoin were gone. Three weeks later it said it had found about 200,000 bitcoin in an old-format wallet file that it believed was empty. Which statement says correctly what each story teaches?

A. The filing: an untested backup file is not an empty one; the old file: whoever holds the keys holds the coins, exchanges included.
B. The filing: customers lost their own keys while writing down a recovery code; the old file: backups expire, so an old wallet file can safely be thrown away.
C. The filing: customers held balances on a website while the keys were in other hands, so whoever holds the keys holds the coins; the old file: a wallet file can still hold spendable keys, so an untested file is not an empty one.
D. The filing: the network let the exchange spend the same coins twice; the old file: miners had held the coins back until the exchange paid a larger fee.

22. A wallet spends one coin of 200,000 satoshis to pay 120,000 satoshis to a payee; everything else returns to the wallet as change. The finished transaction weighs 1,126 weight units and the wallet pays a fee rate of 18 satoshis per virtual byte. How many satoshis does the change output hold?
Sheet row 22, one digit per box. Unit: satoshis.

23. A script lists release tags, drops the release candidates (their tag carries rc), and compares the order of the final releases sorted as text with the order sorted by version number. It adds up how far each tag moves between the two orders. What does it print?

```
tags = ['v28.0', 'v3.1', 'v28.0rc1',
        'v4.0', 'v24.2', 'v9.1']
final = [t for t in tags if 'rc' not in t]
def key(t):
    return [int(p) for p in t[1:].split('.')]
a = sorted(final)
b = sorted(final, key=key)
move = 0
for t in final:
    move += abs(a.index(t) - b.index(t))
print(move)
```

Printed result: sheet row 23, one digit per box.

24. A wallet shows a draft batching payment. Its inputs are 400,000 and 250,000 satoshis. Its outputs are 300,000, 200,000, 100,000 and 29,000 satoshis. A colleague says that every node will refuse it because the inputs and the outputs are not equal. Which reply is correct?

A. Yes: with four outputs the payment is a consolidation, and a consolidation must spend exactly the sum of its inputs.
B. No: the 21,000 satoshis that the outputs leave over are the miner's fee; no field records it, it is the difference between inputs and outputs, so the transaction can be valid.
C. Yes: the ledger must balance exactly, so the wallet must lower one output by 21,000 satoshis before any node will accept the transaction.
D. No: the nodes return the 21,000 satoshis left over to the wallet as change, so nothing is lost and no fee is paid.